

Федеральное государственное бюджетное образовательное учреждение высшего образования «Херсонский государственный педагогический университет»
Институт информационных технологий
Направление подготовки: «09.04.04 Программная инженерия»
Специализация «Разработка программно-информационных систем»
Форма обучения: очная
Курс: 2, Магистратура, группа 12-25РПм
Херсонская область - 2026

Дисциплина:
«Современные операционные системы и базы данных»

Выполнил:
Дмитрий Сергеевич Трифонов
обучающийся 2-го курса

Проверила:
Галлини Н.И.

ПРАКТИЧЕСКАЯ РАБОТА

«Диагностика HTTP сервиса средствами операционной системы»

I. ЦЕЛЬ И ЗАДАЧИ

1.1. Цель

Освоить методы диагностики HTTP-сервиса на разных уровнях: прикладном (HTTP-ответы), транспортном (TCP-сокеты, порты) и уровне ОС (процессы, PID). Научиться различать и воспроизводить состояния «нормальная работа», «ошибка приложения», «недоступность порта» и «тайм-аут», а также автоматизировать проверку работоспособности (healthcheck) с корректным кодом возврата.

1.2. Задачи

- Реализовать локальный HTTP-сервис на Python с использованием ThreadingHTTPServer и набором endpoint'ов для разных сценариев.
- Запустить сервис как отдельный процесс и привязать к 127.0.0.1:8080.
- Сопоставить PID процесса из HTTP-ответа, PID сокета и данные процесса ОС.
- Воспроизвести и задокументировать сценарии: 200 OK, 404 Not Found, 503 Service Unavailable, timeout, connection refused.
- Создать автоматическую проверку healthcheck с корректным exit code.
- Провести сравнительный эксперимент с многопоточным и однопоточным сервером.
- Сформулировать выводы о различиях между уровнями диагностики (ICMP/TCP/HTTP) и границах применимости инструментов.

II. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ

2.1. Программа, процесс, поток, сокет, порт

Программа — это статический исполняемый файл (например, python.exe). Процесс — запущенный экземпляр программы с выделенной памятью и ресурсами; у каждого процесса есть уникальный PID. Поток — единица планирования внутри процесса; ThreadingHTTPServer обрабатывает запросы в отдельных потоках, чтобы не блокировать весь сервер. Сокет — программная конечная точка сетевого обмена; состояние LISTENING означает, что сокет ожидает входящих соединений. Порт — числовой идентификатор конечной точки; комбинация IP:порт однозначно определяет адрес сокета.

2.2. Уровни диагностики и их вопросы

- Уровень ICMP (ping): отвечает на вопрос «достигим ли узел?» — не гарантирует доступность порта или HTTP-сервиса.
- Уровень TCP (Get-NetTCPConnection, netstat): отвечает на вопрос «какой процесс слушает порт и в каком состоянии сокет?»
- Уровень HTTP (curl -i): отвечает на вопрос «какой код состояния, заголовки и тело вернул endpoint?»
- Прикладной уровень (healthcheck): проверяет соответствие контракта (статус 200 + payload.status=ok) и возвращает корректный exit code.

2.3. Connection refused vs Timeout

Connection refused — ОС получила попытку соединения, но на целевом порту нет слушающего сокета (или соединение активно отклонено). Это быстрый отказ с понятным кодом ошибки. Timeout — соединение установлено или находится в процессе установки, но ответ не получен в течение заданного времени. В лабораторной timeout моделируется через медленный endpoint (/slow) и ограничение --max-time у curl.

2.4. Почему 503 подтверждает доступность TCP и HTTP

Код 503 Service Unavailable — это валидный HTTP-ответ, который доказывает, что TCP-соединение установлено, а HTTP-сервер работает и способен формировать ответы. Ошибка относится к уровню приложения (деградация зависимости), а не к недоступности сервиса как такового.

III. МЕТОДЫ И ПОДХОДЫ К ВЫПОЛНЕНИЮ РАБОТЫ

3.1. Подготовка окружения

- Создано виртуальное окружение .venv и активировано в терминале VS Code.
- В папке проекта osdb-http-lab созданы файлы: server.py, check_service.py.
- Для запуска и диагностики использован PowerShell в VS Code.

3.2. Реализация server.py

- Использован ThreadingHTTPServer для многопоточной обработки запросов.
- Endpoint / — возвращает HTML-страницу с навигацией.
- Endpoint /health — возвращает JSON со статусом, PID, портом и uptime.
- Endpoint /slow — имитирует задержку 2 секунды.
- Endpoint /error — возвращает 503 с описанием деградации.
- Все остальные пути — 404.

3.3. Запуск и первичная проверка

- Сервер запущен командой: python server.py
- Проверка корневого адреса: curl.exe -i http://127.0.0.1:8080/ — получен HTML с кодом 200.
- Проверка health: curl.exe -i http://127.0.0.1:8080/health — получен JSON со статусом 200 и PID процесса.

3.4. Связь процесса и порта

- Команда `Get-NetTCPConnection -LocalPort 8080` показывает сокет в состоянии LISTENING и OwningProcess (PID).
- `Get-Process -Id $pidFromPort` выводит данные процесса Python.
- PID из JSON (`/health`), OwningProcess сокета и Id процесса ОС совпадают — это доказывает связь прикладного ответа с конкретным процессом ОС.

3.5. Матрица отказов

- Нормальная работа: `curl -i .../health` → 200 OK и JSON `status=ok`.
- Неизвестный путь: `curl -i .../unknown` → 404 Not Found.
- Деградация: `curl -i .../error` → 503 Service Unavailable.
- Тайм-аут: `curl --max-time 1 .../slow` → ошибка curl 28 (timeout).
- Неверный порт: `curl -i http://127.0.0.1:8081/health` → connection refused.
- Остановленный процесс: после остановки `server.py` повторный запрос → connection refused.

3.6. Автоматизация healthcheck

- `check_service.py` проверяет доступность `http://127.0.0.1:8080/health` с таймаутом 1 секунда.
- Скрипт парсит JSON и убеждается, что `response.status == 200` и `payload["status"] == "ok"`.
- Любые сетевые ошибки, неверный статус или payload приводят к exit code 1.
- Только успешный ответ с корректным статусом даёт exit code 0.

3.7. Дополнительное задание: многопоточность vs однопоточность

- Параллельные запросы к `/slow` при использовании `ThreadingHTTPServer` обрабатываются одновременно — общее время близко к 2 секундам.
- При замене на `HTTPServer` (однопоточный) запросы выполняются последовательно — общее время ~20 секунд.
- Это демонстрирует преимущество многопоточности для блокирующих операций.

IV. ПРАКТИЧЕСКОЕ ВЫПОЛНЕНИЕ И АНАЛИТИЧЕСКОЕ ОСМЫСЛЕНИЕ

4.1. Реализованный код и запуск

- Файл `server.py` содержит реализацию всех требуемых endpoint'ов и использует `ThreadingHTTPServer`.
- Сервер привязан к `127.0.0.1:8080`, что ограничивает доступ локальной машиной.
- Запуск в первом терминале VS Code; второй терминал используется для диагностики.

4.2. Сопоставление PID на разных уровнях

- Из ответа `/health` получен PID процесса.
- Через `Get-NetTCPConnection` определён `OwningProcess` для порта `8080` — значение совпало.
- Через `Get-Process` подтверждены данные процесса (имя, CPU, `WorkingSet`, `StartTime`).
- Совпадение трёх значений доказывает однозначную связь между HTTP-ответом, сокетом и процессом ОС.

4.3. Результаты матрицы отказов

- 200 OK: корректный ответ `healthcheck` — сервис готов.
- 404: TCP и HTTP работают, запрошенный ресурс отсутствует.
- 503: сервис доступен, но зависимость недоступна — это деградация, а не отказ соединения.
- Timeout: клиент прекратил ожидание из-за медленного ответа.
- Connection refused: на порту нет слушающего сокета — либо порт неверен, либо сервер остановлен.

4.4. Автоматизированный healthcheck

- `check_service.py` корректно возвращает 0 при здоровом сервисе.
- При сетевых сбоях, неверном статусе или payload возвращается 1.
- Такой скрипт можно использовать в CI/CD и оркестраторах для проверки готовности сервиса.

4.5. Сравнение многопоточного и однопоточного сервера

- ThreadingHTTPServer позволяет обрабатывать запросы параллельно — задержка /slow не блокирует другие запросы.
- HTTPServer обрабатывает запросы последовательно — при 10 параллельных запросах общее время линейно растёт.
- Вывод: для реальных сервисов с блокирующими операциями предпочтительна многопоточная или асинхронная модель.

4.6. Ответы на вопросы защиты (кратко)

- PID меняется после перезапуска, потому что ОС назначает новый уникальный идентификатор каждому новому процессу.
- Сокет — это программная абстракция (конечная точка), а порт — числовой адрес; один порт может иметь несколько сокетов (IPv4/IPv6).
- Состояние LISTENING доказывает, что процесс открыл сокет и готов принимать соединения на указанном порту.
- Ping проверяет ICMP, а не TCP/HTTP — успешный ping не гарантирует, что порт 8080 открыт или HTTP-сервис работает.
- Connection refused — быстрый отказ ОС; timeout — отсутствие ответа в заданное время.
- 503 — это HTTP-ответ, значит TCP-соединение и HTTP-сервер работают, проблема на уровне приложения.
- Exit code healthcheck используется оркестраторами (Kubernetes, Docker Swarm) для определения готовности контейнера.
- Однопоточный сервер полностью блокируется во время выполнения /slow — другие запросы ждут своей очереди.
- Привязка к 0.0.0.0 делает сервис доступным из сети — требуются правила фаервола и меры безопасности.
- В production-журналы нельзя записывать персональные данные, пароли, токены и чувствительные параметры запросов.

V. ВЫВОДЫ

В ходе работы был реализован локальный HTTP-сервис с набором endpoint'ов, моделирующих различные сценарии работы и отказов. Удалось чётко продемонстрировать связь между прикладным уровнем (HTTP-коды и JSON), транспортным уровнем (TCP-сокеты и порты) и уровнем ОС (процессы и PID). Были воспроизведены и задокументированы состояния нормальной работы, деградации, тайм-аута и недоступности порта. Реализован автоматизированный healthcheck с корректным кодом возврата, пригодный для использования в CI/CD. Эксперимент с многопоточным и однопоточным серверами наглядно показал влияние модели обработки запросов на производительность при блокирующих операциях. Работа подтвердила, что для комплексной диагностики сервиса необходимо применять инструменты на разных уровнях и понимать границы их применимости.

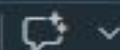
main.py X first.py

main.py > ...

```
2 from fastapi import FastAPI
3
4 app = FastAPI()
5
6 @app.get("/")
7 def read_root():
8     return {"message": "Учебный HTTP-сервис работает", "pid": os.getpid()}
9
10 @app.get("/health")
11 def health():
12     return {"status": "ok", "pid": os.getpid()}
13
```



20260917 [Administrator]



127.0.0.1:8000 X

← → ↻ http://127.0.0.1:8000/

Pretty-print ☐

```
{"message": "Учебный HTTP-сервис работает", "pid": 6512}
```

```
mitriy@DESKTOP-0NVCEI5 MINGW64 ~/Desktop/20260917
```

```
$ curl -i http://127.0.0.1:8000/docs
```

```
● HTTP/1.1 200 OK
```

```
date: Thu, 17 Sep 2026 10:19:17 GMT
```

```
server: uvicorn
```

```
content-length: 1006
```

```
content-type: text/html; charset=utf-8
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<link type="text/css" rel="stylesheet" href="https://cdn.jsdelivrivr.net/npm/swagger-ui-dist@5/swagger-ui.css">
```

```
<link rel="shortcut icon" href="https://fastapi.tiangolo.com/img/favicon.png">
```

```
<title>FastAPI - Swagger UI</title>
```

```
</head>
```

```
<body>
```

```
<div id="swagger-ui">
```

```
</div>
```

```
<script src="https://cdn.jsdelivrivr.net/npm/swagger-ui-dist@5/swagger-ui-bundle.js"></script>
```

```
<!-- `SwaggerUIBundle` is now available on the page -->
```

```
<script>
```

```
const ui = SwaggerUIBundle({
```

```
  url: '/openapi.json',
```

```
  "dom_id": "#swagger-ui",
```

```
mitriy@DESKTOP-0NVCEI5 MINGW64 ~/Desktop/20260917
$ curl -i http://127.0.0.1:8000/
date: Thu, 17 Sep 2026 10:18:31 GMT
server: uvicorn
content-length: 124
content-type: application/json

{"message": "Здравствуйтесь, Надежда Игоревна! Сервер работает. Что дальше?"}(venv)
mitriy@DESKTOP-0NVCEI5 MINGW64 ~/Desktop/20260917
```

 MINGW64:/c/Users/mitriy

mitriy@DESKTOP-ONVCEI5 MINGW64 ~

\$ netstat -ano | findstr :8000

TCP	127.0.0.1:8000	0.0.0.0:0	LISTENING	8068
-----	----------------	-----------	-----------	------

mitriy@DESKTOP-ONVCEI5 MINGW64 ~

\$ |

Pretty-print ☐

```
{"message": "Здравствуйте, Надежда Игоревна! Сервер работает. Что дальше?"}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(venv)

mitriy@DESKTOP-0NVCEI5 MINGW64 ~/Desktop/20260917

\$ uvicorn main:app --reload --log-level debug

INFO: Started reloader process [8068] using StatReload

INFO: Started server process [6596]

INFO: Waiting for application startup.

INFO: Application startup complete.

INFO: 127.0.0.1:53377 - "GET / HTTP/1.1" 200 OK